

ParkEase — Complete Updated Antigravity Master Prompt

PARKEASE — COMPLETE UPDATED ANTIGRAVITY MASTER DEVELOPMENT PROMPT

IMPORTANT:

You are working on an existing ParkEase Smart Parking System major project.
First inspect the complete repository and understand the existing implementation.
Do NOT rewrite the project from scratch unless absolutely necessary.

1. PROJECT OBJECTIVE

Build a complete Smart Parking System using:

Frontend → Vercel
Backend → Flask on Render or Railway
Database → Hosted MySQL
Hardware → ESP32 + IR sensors + Servo

The system must support:

- User registration/login
- User parking dashboard
- Slot booking
- Automatic user profile data
- Vehicle entry authorization
- ESP32 gate control
- Parking slot occupancy detection
- Vehicle exit
- Admin login
- Protected admin dashboard
- Live parking monitoring
- Booking and vehicle management
- Parking statistics
- Sensor/gate monitoring

Backend/database must remain authoritative.

2. AUTHENTICATION-FIRST LANDING PAGE — REQUIRED

The application MUST NOT show the main parking dashboard immediately to an unauthenticated visitor.

The FIRST screen must be a Welcome / Authentication screen.

Show:

PARKEASE
Smart Parking System

[LOGIN]

[CREATE ACCOUNT]

Administrator?

[ADMIN LOGIN]

The page must be modern, attractive, responsive and consistent with the existing ParkEase branding.

3. USER FLOW

Visitor opens website
↓
Welcome / Authentication Screen
↓
Login OR Register
↓
Authenticated User
↓
User Parking Dashboard

After authentication, the user can:

- View live parking slots

- Book a parking slot
- View My Booking
- Enter Parking
- Cancel booking
- View account/profile
- Logout

Unauthenticated visitors must not access the parking dashboard or booking functions.

=====

4. USER REGISTRATION

=====

Registration fields:

- Full Name
- Email
- Mobile Number
- Password
- Confirm Password

Requirements:

- Secure password hashing
- Never store plain-text passwords
- Backend validation
- Email validation
- Mobile validation
- Password validation
- Duplicate account prevention

After successful registration, either automatically authenticate the user or redirect them to Login, using a clean UX.

=====

5. USER LOGIN

=====

Login fields:

- Email
- Password

After successful authentication:

→ Redirect to User Parking Dashboard.

Authentication must be backend-authoritative.

Do NOT rely on localStorage alone to determine authentication.

=====

6. BOOKING FORM – IMPORTANT NEW REQUIREMENT

=====

When an authenticated user books a parking slot, DO NOT ask them to enter their name and mobile number again.

The user's account information already exists in the database.

The booking form must automatically fetch the logged-in user's information from the backend/database.

Booking form:

Full Name
→ Automatically fetched from database
→ Read-only / disabled

Mobile Number
→ Automatically fetched from database
→ Read-only / disabled

Vehicle Number
→ ONLY field the user manually enters
→ Required
→ Validate appropriately

Selected Slot
→ Automatically populated from selected slot
→ Read-only
→ Backend validates availability again

Example:

BOOK YOUR PARKING SLOT

Full Name
[Sahil Kajare]

Mobile Number
[98XXXXXXX]

Vehicle Number
[MH12AB1234]

Selected Slot
[S1]

[CONFIRM BOOKING]

The user should only have to enter the vehicle number.

7. BOOKING SECURITY

Do NOT trust name, mobile number or user ID supplied by the frontend.

Preferred booking request:

POST /api/book

```
{
  "slot_id": 1,
  "vehicle_number": "MH12AB1234"
}
```

The frontend should NOT need to send:

- full_name
- mobile
- user_id

Backend process:

```
Authenticated Session/Token
↓
Identify current User
↓
Fetch User from Database
↓
Fetch/lock selected Parking Slot
↓
Validate availability
↓
Create Booking
↓
Mark slot RESERVED
```

The backend must determine the current user from authentication.

A user must never be able to impersonate another user by modifying frontend data.

8. MY BOOKING

My Booking must show bookings belonging only to the authenticated user.

Never trust a frontend-supplied user ID.

Backend must identify the current user from authentication.

Show:

- Booking ID
- Vehicle number
- Slot
- Status
- Booking time
- Entry time
- Exit time
- Enter Parking button when authorized
- Cancel button when applicable

=====

9. ADMIN LOGIN

=====

There must be a separate Admin Login option on the first screen.

Admin Login:

Admin Email
Admin Password

[LOGIN]

[← Back to User Login]

There must be NO public Admin Registration.

=====

10. ADMIN AUTHENTICATION

=====

Admin credentials must be configured server-side.

Use environment variables such as:

ADMIN_EMAIL
ADMIN_PASSWORD_HASH

Never expose admin credentials in:

- HTML
- CSS
- JavaScript
- Vercel frontend
- GitHub
- public configuration

Backend must verify admin credentials.

Use a secure authentication/session mechanism.

=====

11. ADMIN AUTHORIZATION

=====

Roles:

USER
ADMIN

Normal users:

- Cannot access admin dashboard
- Cannot access admin APIs

Admins:

- Can access admin dashboard
- Can access protected admin APIs

Frontend checks alone are NOT sufficient.

Backend must enforce authorization.

Directly opening an admin URL without authentication must be rejected.

A normal user calling an admin API must receive 401/403.

=====

12. ADMIN DASHBOARD

=====

Admin dashboard sections:

1. Dashboard Overview
2. Live Parking
3. All Bookings
4. Active Vehicles
5. Parking Statistics
6. Sensors & Gate Status
7. Logout

Overview cards:

- Total Slots
- Available Slots
- Reserved Slots
- Occupied Slots
- Exiting Slots
- Active Bookings
- Active Vehicles
- Today's Bookings
- Today's Completed Exits

All data must come from backend/database.

Do not hardcode statistics.

13. LIVE PARKING

Display S1, S2 and S3 as visual parking cards.

Each card:

- Slot ID
- Current status
- Sensor status
- Vehicle number if available

Statuses:

AVAILABLE
RESERVED
OCCUPIED
EXITING

Refresh automatically.

14. ALL BOOKINGS – ADMIN

Show:

- Booking ID
- User
- Email
- Mobile
- Vehicle
- Slot
- Booking Status
- Entry Time
- Exit Time
- Created At

Provide:

- Search
- Filtering
- Sorting
- Responsive layout
- Pagination if useful

Never show passwords.

15. ACTIVE VEHICLES

Show vehicles currently inside:

- Vehicle Number
- User
- Slot
- Entry Time
- Current Status

Data must come from database.

16. PARKING STATISTICS

Provide:

- Total bookings
- Available vs occupied
- Daily bookings

- Completed sessions
- Active vehicles
- Slot utilization

Use charts where they improve understanding.

=====

17. SENSOR & GATE MONITORING

=====

Admin must see:

Entry Sensor
Exit Sensor
S1 Sensor
S2 Sensor
S3 Sensor

Values:
EMPTY / VEHICLE DETECTED

Also:
Gate OPEN / CLOSED
Movement IDLE / ENTERING / EXITING
Pending Entry Booking
Pending Exit Slot
Last IoT update/time

This is important for demonstrating the ESP32 integration.

=====

18. HARDWARE

=====

ESP32 pins:

Entry IR → GPIO18
Exit IR → GPIO19
Slot 1 IR → GPIO21
Slot 2 IR → GPIO22
Slot 3 IR → GPIO23
Servo → GPIO25

IR sensors are active LOW.

Servo:
- External 5V supply
- Common GND with ESP32
- Signal on GPIO25

Do not power the servo from ESP32 3.3V.

=====

19. PHYSICAL LAYOUT

=====

Road / Car
↓
Entry IR
↓
Servo Gate
↓
Exit IR
↓
Parking Area

=====

20. ENTRY LOGIC

=====

Entry IR detects a waiting vehicle.

Entry IR alone MUST NOT automatically open the gate.

User:
1. Logs in
2. Books slot
3. Vehicle reaches Entry IR
4. User taps ENTER PARKING
5. Backend verifies authorization

6. ESP32 receives authorization
7. Servo opens
8. Vehicle crosses gate
9. Exit IR detects vehicle
10. ESP32 reports entry crossing
11. Gate closes
12. Slot sensor detects parked vehicle
13. Slot becomes OCCUPIED

21. EXIT LOGIC

Exit IR detects vehicle leaving.

1. Exit sequence begins.
2. Gate opens.
3. Occupied slot that becomes empty is identified.
4. Slot becomes EXITING.
5. EXITING remains unavailable.
6. Entry IR confirms vehicle has completely exited.
7. Gate closes.
8. EXITING → AVAILABLE.

With only five sensors, exact source slot cannot always be determined from Exit alone.

For this prototype, identify the occupied slot whose sensor changes OCCUPIED → EMPTY during the active exit sequence.

22. STATE MACHINE

Movement states:

IDLE
ENTERING
EXITING

Slot states:

AVAILABLE
RESERVED
OCCUPIED
EXITING

Sensor states:

EMPTY
OCCUPIED

Normal lifecycle:

AVAILABLE
→ RESERVED
→ OCCUPIED
→ EXITING
→ AVAILABLE

Entry:

IDLE → ENTERING → IDLE

Exit:

IDLE → EXITING → IDLE

Avoid contradictory states.

23. BACKEND

Use Flask.

Backend responsibilities:

- Authentication
- User management
- Booking
- Parking state

- IoT APIs
- Admin APIs
- Authorization
- Transactions
- Concurrency-safe booking
- CORS
- Environment configuration

ESP32 must NEVER connect directly to MySQL.

24. API STRUCTURE

Existing APIs to preserve where already implemented:

```
GET /api/slots
POST /api/book
POST /api/cancel
POST /api/enter-parking
POST /api/iot/entry
GET /api/iot/entry-status
GET /api/entry-status
POST /api/iot/slots
POST /api/iot/exit
POST /api/iot/entry-crossed
```

Authentication APIs:

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me
```

Admin APIs as appropriate:

```
POST /api/admin/login
POST /api/admin/logout
GET /api/admin/dashboard
GET /api/admin/bookings
GET /api/admin/vehicles
GET /api/admin/statistics
GET /api/admin/iot-status
```

Inspect the existing repository before changing endpoint names.

25. DATABASE

Keep MySQL unless there is a strong technical reason to migrate.

Important entities:

- User
- Booking
- ParkingSlot
- ParkingSystemState

Inspect existing schema/models before modifying.

Booking creation must be concurrency-safe.

Two users cannot successfully reserve the same active slot.

Use transactions/row locking appropriately.

Do not hide all IntegrityError causes behind an unexplained generic error. Log/diagnose the actual database cause safely.

26. FRONTEND

Preserve existing ParkEase UI and functionality.

Existing features:

- ParkEase branding
- Navbar
- Hero
- Parking section
- Live parking status

- Booking modal
- My Booking
- Enter Parking
- Cancel reservation
- Dark mode
- Responsive design

Integrate authentication without unnecessarily destroying the existing UI.

Backend/database remains the source of truth.

27. AUTHENTICATION STATE

On page load:

1. Check backend authentication/session.
2. Authenticated user → User Dashboard.
3. Authenticated admin → Admin Dashboard.
4. Not authenticated → Welcome/Auth screen.

Do not trust stale localStorage.

28. ACCOUNT PAGE

User account page can show:

- Full Name
- Email
- Mobile Number
- Registered vehicles if implemented
- Logout

Profile information must come from backend/database.

29. UI / UX

Create a polished modern application.

Authentication:

- ParkEase branding
- Attractive visual
- Login
- Register
- Admin Login
- Smooth transitions

User dashboard:

- Live slot cards
- Booking controls
- My Booking
- Enter Parking
- Cancel
- Account
- Logout

Admin:

- Professional dashboard
- Sidebar/navigation
- Cards
- Tables
- Charts
- IoT monitoring

30. RESPONSIVE DESIGN

Test:

- 320px
- 375px
- 390px
- 430px
- 768px
- 1024px

1440px+

No:

- horizontal scrolling
- overlap
- broken tables
- broken modals
- text overflow
- inaccessible buttons

31. ANIMATIONS

Use tasteful:

- Page transitions
- Login/register transitions
- Card hover
- Button effects
- Slot status transitions
- Loading states
- Toast notifications
- Sensor transitions

Do not overuse animations.

Support prefers-reduced-motion.

32. DEPLOYMENT

Target:

Frontend → Vercel
Backend → Render or Railway
Database → Hosted MySQL
ESP32 → Deployed Flask API

Do not hardcode production:

localhost
127.0.0.1
10.81.231.98

Use environment variables:

DATABASE_URL
SECRET_KEY
IOT_API_KEY
ADMIN_EMAIL
ADMIN_PASSWORD_HASH
FRONTEND_URL

Never expose secrets in frontend code.

33. CORS

Configure CORS for:

- Vercel production frontend
- Local development

If cookies/sessions are used, configure credentials and allowed origins correctly.

34. REPOSITORY INSPECTION – REQUIRED

Before changing anything:

1. Inspect complete repository.
2. Identify frontend entry point.
3. Identify Flask entry point.
4. Inspect models.
5. Inspect services.
6. Inspect routes.
7. Inspect authentication.
8. Inspect database configuration.
9. Inspect CSS.
10. Inspect JavaScript.

11. Inspect ESP32 code.
12. Inspect environment configuration.
13. Inspect deployment configuration.
14. Determine which code version is actually running.

There have previously been conflicting versions of backend service code. Verify actual imports/running code instead of assuming.

```
=====
35. DO NOT ASK FOR MANY MANUAL PATCHES
=====
```

Make coherent changes across all affected files.

Do not give instructions such as:
"change these five lines manually."

Instead:

- inspect
- implement
- update all affected files
- maintain imports
- maintain API compatibility
- maintain database consistency
- maintain ESP32 compatibility
- test

```
=====
36. TESTING – USER
=====
```

- [] First screen is Welcome/Auth
- [] Dashboard hidden before login
- [] Registration works
- [] Duplicate account rejected
- [] Login works
- [] Wrong password rejected
- [] Logout works
- [] Authenticated dashboard loads
- [] Live slots load
- [] Booking works
- [] Name automatically fetched
- [] Mobile automatically fetched
- [] Only vehicle number needs manual entry
- [] My Booking works
- [] Enter Parking works
- [] Cancellation works

```
=====
37. TESTING – ADMIN
=====
```

- [] Admin Login visible on first screen
- [] No Admin Registration
- [] Correct admin credentials work
- [] Incorrect credentials rejected
- [] Admin dashboard protected
- [] Normal user blocked from admin
- [] Normal user blocked from admin APIs
- [] Live parking works
- [] Bookings work
- [] Active vehicles work
- [] Statistics work
- [] Sensor/gate status works
- [] Admin logout works

```
=====
38. TESTING – IOT
=====
```

- [] Entry IR
- [] Exit IR
- [] S1
- [] S2
- [] S3
- [] Servo
- [] Entry authorization
- [] Entry crossing
- [] Slot occupancy
- [] Exit authorization
- [] EXITING state
- [] Slot release

39. FINAL USER EXPERIENCE

Visitor:

PARKEASE

Smart Parking System

↓

Welcome / Login / Register / Admin Login

USER:

Register/Login

↓

User Parking Dashboard

↓

Live Slots

↓

Select Slot

↓

Booking Form

↓

Name + Mobile automatically fetched

↓

User enters Vehicle Number only

↓

Confirm Booking

↓

My Booking

↓

ENTER PARKING

↓

ESP32 + IR Sensors + Servo

↓

Parking

↓

Exit

ADMIN:

Admin Login

↓

Admin Dashboard

↓

Overview

↓

Live Parking

↓

Bookings

↓

Active Vehicles

↓

Statistics

↓

Sensors & Gate

↓

Logout

40. FINAL REPORT

After implementation provide:

1. Files changed
2. Features added
3. Database changes
4. API endpoints
5. Authentication approach
6. User registration/login flow
7. Automatic user-data fetching flow
8. Admin authentication
9. Admin authorization
10. UI improvements
11. ESP32 changes
12. Environment variables
13. Local run commands
14. Vercel deployment steps
15. Render/Railway deployment steps
16. Database setup/migration
17. Testing results
18. Remaining issues

Do not claim anything works unless actually verified.

START NOW:

First inspect the existing repository and understand what is already implemented.

Then implement the complete authentication-first architecture, including user registration/login, automatic database-based user details in booking, vehicle-number-only booking input, separate admin login/dashboard, protected authorization, and all existing ParkEase parking/ESP32 functionality.

Do not break existing working functionality.